Adaptive Runge–Kutta–Fehlberg Method (RKF45) vs Classical RK4 Applied to the Mass–Spring–Damper System

Aman Pradhan
Roll No. 2311020
P346-DIY PROJECT

School of Physical Sciences, NISER

2025

Outline

1. Introduction
2. Mass–Spring–Damper System
3. Analytical Solution
4. Fehlberg Method (RKF45)
5. Results
6. Discussion & Conclusion
7. Outlook
8. References

Ordinary Differential Equations

- Ordinary differential equations arise throughout physics, chemistry, biology, and engineering, and many systems cannot be solved analytically.
- Classical fixed–step schemes like RK4 provide good accuracy, but only when the step size is sufficiently small. For rapidly oscillating or stiff systems, RK4 either becomes inaccurate or requires extremely small steps, increasing computational cost.
- Fehlberg's method (RKF45) overcomes this limitation through an **embedded Runge–Kutta pair**: a 4th-order method and a 5th-order error estimator computed using the same intermediate slopes.
- Because two solutions of different accuracy are obtained simultaneously, the local truncation error can be estimated at each step without extra function calls.
- This estimated error allows the algorithm to **automatically adjust the step size**: increasing it when the solution is smooth and decreasing it when rapid changes occur.
- The adaptive behaviour of RKF45 provides significantly higher efficiency and stability compared with fixed-step RK4 for oscillatory or nonlinear systems.

Mathematical Model and Physical Interpretation

Why choose the mass–spring–damper system?

- It is one of the most fundamental models describing oscillatory motion in physics and engineering.
- Equivalent analogs appear in electrical RLC circuits, molecular vibrations, biomechanics, and control systems.
- It contains restoring, damping, and forcing effects—making the dynamics rich enough to test numerical solvers.
- The system exhibits regions of both slow and rapid variation, ideal for demonstrating why **adaptive RKF45** outperforms fixed-step RK4.

Deriving the governing equation:

$$mx''(t) = F_{\text{total}} = -kx(t) \; - \; ax'(t) \; + \; F_0 \sin(\omega t)$$

Rearranging:

$$mx''(t) + ax'(t) + kx(t) = F_0 \sin(\omega t)$$

Physical interpretation:

- Spring force: $F_s = -kx$ (restoring)
- Damping force: $F_d = -ax'$ (energy dissipation)
- External periodic forcing: $F_0 \sin(\omega t)$
- Total motion = *transient decay* + *steady-state forced oscillation*

The transient part dies out due to damping, while the periodic forcing sustains oscillations at long times.

Conversion to First-Order System

Numerical Runge–Kutta methods (RK4, RKF45) require a system of first-order ODEs. Hence, the second-order equation

$$mx''(t) + ax'(t) + kx(t) = F_0 \sin(\omega t)$$

must be rewritten in first-order form.

Step 1: Introduce state variables

$$x(t) = \text{displacement}, \qquad v(t) = x'(t) = \text{velocity}$$

Step 2: Express derivatives

$$x'(t) = v$$

$$v'(t) = \frac{F_0 \sin(\omega t) - av - kx}{m}$$

Step 3: System form for numerical integration Define

$$y(t) = \begin{bmatrix} x(t) \\ v(t) \end{bmatrix}, \qquad y'(t) = f(t, y) = \begin{bmatrix} v \\ \dfrac{F_0 \sin(\omega t) - av - kx}{m} \end{bmatrix}.$$

Why this form is required:

- RK4 and RKF45 evaluate $f(t, y)$ repeatedly at shifted points.
- They operate only on first-order systems.
- Converting the system ensures compatibility with all Runge–Kutta methods.

Thus, the mass–spring–damper dynamics become a 2-dimensional first-order ODE system suitable for adaptive time stepping.

General Analytical Form(1/2)

We solve the forced damped oscillator:

$$mx'' + ax' + kx = F_0 \sin(\omega t)$$

General solution:

$$x(t) = x_h(t) + x_p(t)$$

1. Homogeneous solution
Solve:

$$mx'' + ax' + kx = 0$$

Characteristic equation:

$$mr^2 + ar + k = 0$$

$$r = \frac{-a \pm \sqrt{a^2 - 4mk}}{2m}$$

For the underdamped case ($a^2 < 4mk$):

$$r = -\frac{a}{2m} \pm i\sqrt{\frac{k}{m} - \left(\frac{a}{2m}\right)^2}$$

Define

$$\alpha = -\frac{a}{2m}, \qquad \beta = \sqrt{\frac{k}{m} - \alpha^2}$$

Thus,

$$x_h(t) = e^{\alpha t}\left[C\cos(\beta t) + D\sin(\beta t)\right]$$

which captures the *transient*, exponentially decaying oscillation.

General Analytical Form(2/2)

2. Particular (forced) solution
Assume sinusoidal steady-state motion:

$$x_p(t) = A\sin(\omega t) + B\cos(\omega t)$$

Compute derivatives:

$$x_p' = A\omega\cos(\omega t) - B\omega\sin(\omega t)$$

$$x_p'' = -A\omega^2\sin(\omega t) - B\omega^2\cos(\omega t)$$

Substitute into ODE and collect terms in $\sin\omega t$ and $\cos\omega t$. Matching coefficients yields:

$$A = \frac{F_0(k-\omega^2)}{(k-\omega^2)^2 + (a\omega)^2}, \qquad B = -\frac{F_0(a\omega)}{(k-\omega^2)^2 + (a\omega)^2}$$

This provides the *steady-state* sinusoid with phase lag.

3. Interpretation of the plot

- The early part shows the decaying exponential $e^{\alpha t}$ from $x_h(t)$: the **transient dies out**.
- After decay, only $x_p(t)$ remains: a pure sinusoid at the forcing frequency ω.
- This explains why the displacement plot transitions from large, irregular oscillations to a smooth fixed-amplitude sinusoid.

Flow of Numerical Solution (RK4 / RKF45)

Start with 2nd-order ODE:

$$mx'' + ax' + kx = F_0 \sin(\omega t)$$

Convert to 1st-order system:

$$y = \begin{bmatrix} x \\ v \end{bmatrix}, \quad y' = f(t, y) = \begin{bmatrix} v \\ \frac{F_0 \sin(\omega t) - av - kx}{m} \end{bmatrix}$$

For each time step: evaluate $f(t_n, y_n)$

RK4: compute k_1, k_2, k_3, k_4
update: $y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$

RKF45: compute $k_1, \ldots, k_6$
compute $y_{n+1}^{(4)}, y_{n+1}^{(5)}$
estimate error $\epsilon = |y^{(5)} - y^{(4)}|$

Adaptive step-size:

$$h_{\text{new}} = 0.9\, h \left(\frac{\text{tol}}{\epsilon}\right)^{1/4}$$

Repeat until $t = t_{\max}$

Classical RK4 Algorithm (Derived from Python Implementation)

Given the first–order system

$$y' = f(t, y), \qquad y(t) = \begin{bmatrix} x \\ v \end{bmatrix},$$

the Python RK4 routine implements the following steps for each time step h:

1. Compute slopes:

$$k_1 = f(t_n,\ y_n),$$
$$k_2 = f\left(t_n + \frac{h}{2},\ y_n + \frac{h}{2}k_1\right),$$
$$k_3 = f\left(t_n + \frac{h}{2},\ y_n + \frac{h}{2}k_2\right),$$
$$k_4 = f(t_n + h,\ y_n + hk_3).$$

2. RK4 update used in the Python code:

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4).$$

3. Time is advanced:

$$t_{n+1} = t_n + h.$$

4. Components stored:

$$x_{n+1} = y_{n+1}(1), \qquad v_{n+1} = y_{n+1}(2).$$

RKF45 (I): Butcher coefficients and rationale

Fehlberg's RKF45 is an *embedded* Runge–Kutta pair: a 5th-order formula and a 4th-order formula that share the same stage evaluations. The method reuses intermediate slopes $k_1, \ldots, k_6$ so the error estimate comes at very little extra cost.

Fehlberg (Cash–Karp / Fehlberg-style) coefficients used in code:

$$a_2 = \tfrac{1}{4},\ a_3 = \tfrac{3}{8},\ a_4 = \tfrac{12}{13},\ a_5 = 1,\ a_6 = \tfrac{1}{2}$$

$$\begin{aligned}
b_{21} &= \tfrac{1}{4}, \\
b_{31} &= \tfrac{3}{32},\ b_{32} = \tfrac{9}{32}, \\
b_{41} &= \tfrac{1932}{2197},\ b_{42} = -\tfrac{7200}{2197},\ b_{43} = \tfrac{7296}{2197}, \\
b_{51} &= \tfrac{439}{216},\ b_{52} = -8,\ b_{53} = \tfrac{3680}{513},\ b_{54} = -\tfrac{845}{4104}, \\
b_{61} &= -\tfrac{8}{27},\ b_{62} = 2,\ b_{63} = -\tfrac{3544}{2565},\ b_{64} = \tfrac{1859}{4104},\ b_{65} = -\tfrac{11}{40}.
\end{aligned}$$

Why these numbers?

- The coefficients satisfy the order conditions for an embedded 5(4) pair: algebraic constraints derived from matching Taylor expansions up to $O(h^5)$.
- Different published RKF variants (Fehlberg, Cash–Karp, Dormand–Prince) pick slightly different coefficients to optimise stability or error constants.
- Using an embedded pair means *no extra function evaluations* are needed to produce both the higher- and lower-order estimates — crucial for efficiency.

RKF45 (II): Stage computations $k_1, \ldots, k_6$

For a first-order system $y' = f(t, y)$ the six stages are computed as:

$$k_1 = f(t_n,\, y_n),$$

$$k_2 = f(t_n + a_2 h,\; y_n + h\, b_{21} k_1)\,,$$

$$k_3 = f(t_n + a_3 h,\; y_n + h\,(b_{31} k_1 + b_{32} k_2))\,,$$

$$k_4 = f(t_n + a_4 h,\; y_n + h\,(b_{41} k_1 + b_{42} k_2 + b_{43} k_3))\,,$$

$$k_5 = f(t_n + a_5 h,\; y_n + h\,(b_{51} k_1 + b_{52} k_2 + b_{53} k_3 + b_{54} k_4))\,,$$

$$k_6 = f(t_n + a_6 h,\; y_n + h\,(b_{61} k_1 + b_{62} k_2 + b_{63} k_3 + b_{64} k_4 + b_{65} k_5))\,.$$

Notes on the algebra:

- Each stage is an evaluation of the vector right-hand side f at a carefully chosen time and state point. The linear combinations of previous k-values approximate higher derivatives in the Taylor expansion.
- The stage times $t_n + a_i h$ are fractional offsets within the step, chosen so the resulting linear system meets the order conditions.
- In the Python code each k_j is computed as `f(t + a*h, [y[i] + h*(...  )])`, which maps exactly to the formulas above.

Practical remark: evaluate and store k_j as vectors (size $= \dim(y)$) and reuse them for both embedded estimates; this is the efficiency gain.

RKF45 (III): Embedded estimates and error estimate

After computing $k_1, \ldots, k_6$ we form the embedded estimates:

$$y_{n+1}^{(4)} = y_n + h \sum_{j=1}^{6} c_j^* \, k_j, \qquad y_{n+1}^{(5)} = y_n + h \sum_{j=1}^{6} c_j \, k_j,$$

with

$$c^* = \left(\tfrac{25}{216}, 0, \tfrac{1408}{2565}, \tfrac{2197}{4104}, -\tfrac{1}{5}, 0\right), \quad c = \left(\tfrac{16}{135}, 0, \tfrac{6656}{12825}, \tfrac{28561}{56430}, -\tfrac{9}{50}, \tfrac{2}{55}\right).$$

Local error estimate: the code uses the infinity norm (max over components)

$$\epsilon \;=\; \max_i \left| \, y_{n+1,i}^{(5)} - y_{n+1,i}^{(4)} \, \right|.$$

Why the embedded difference estimates the error?

- $y^{(5)}$ approximates the true solution to $O(h^6)$ local truncation, while $y^{(4)}$ is accurate to $O(h^5)$.
- The subtraction cancels lower-order terms, leaving an estimate of the leading omitted term — a practical proxy for the local truncation error.

Acceptance criterion (Python):

$$\text{accept step if} \quad \epsilon \leq \text{tol}.$$

Choice of norm and tol:

- The infinity norm is robust and cheap; for vector problems one may use a weighted norm combining absolute and relative tolerances.
- `tol` in code is a user parameter (e.g. 10^{-6}); choose it according to problem scale and desired accuracy.

RKF45 (IV): Step-size control, accept/reject, storage

Step-size control:

$$s = \begin{cases} 2, & \epsilon = 0, \\ 0.9\left(\dfrac{\text{tol}}{\epsilon}\right)^{1/4}, & \epsilon > 0, \end{cases} \qquad h_{\text{new}} = h\,\min\big(\max(s, 0.1), 4.0\big).$$

Derivation and rationale:

- For a method of local order p (here effectively $p=4$), the leading local error scales like Ch^{p+1}.
- To make the new error $\approx$ tol we set $h_{\text{new}} = h\,(\text{tol}/\epsilon)^{1/(p+1)}$. For $p = 4$ this exponent is $1/5$, but practitioners apply a safety factor and use $1/4$ in some implementations to be conservative for embedded pairs; the Python code uses exponent $1/4$ together with a safety factor 0.9.
- Lower/upper bounds (0.1, 4.0) prevent steps from changing too aggressively.

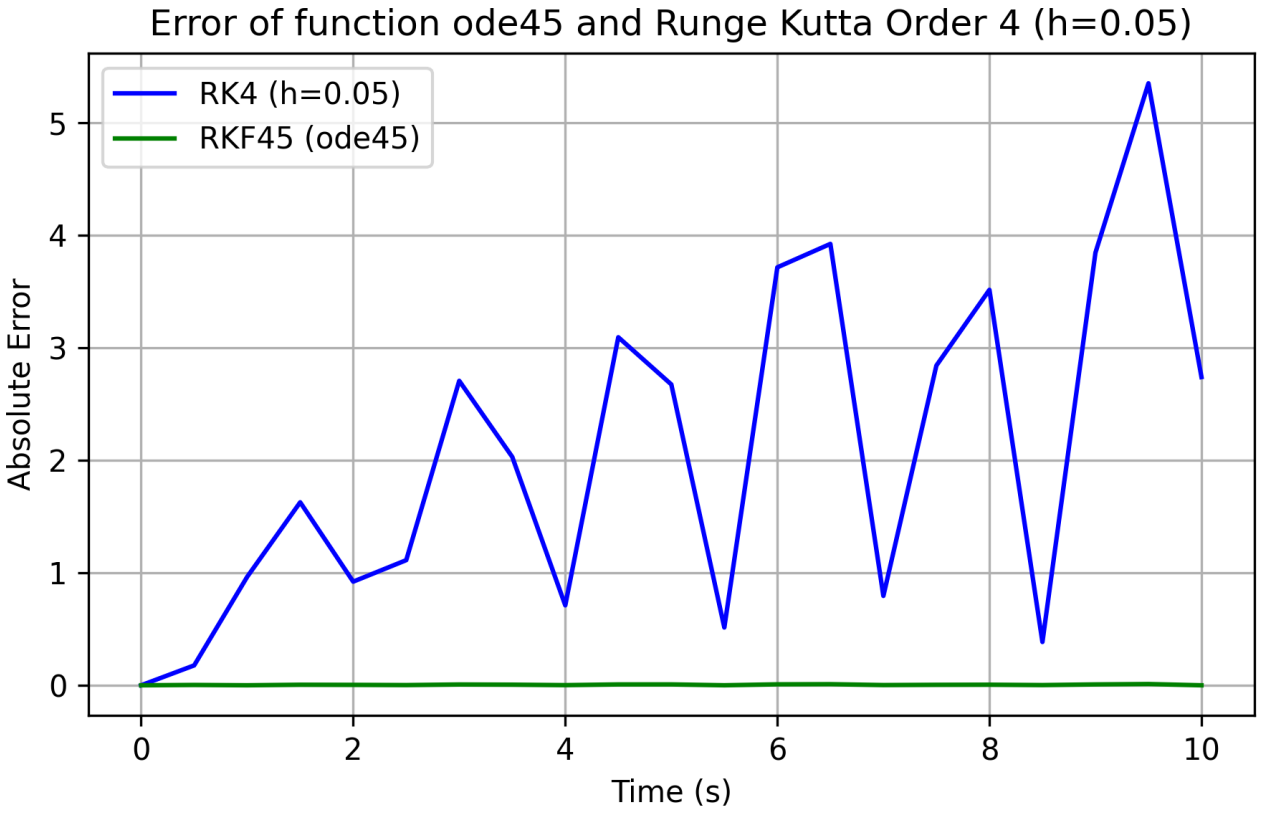
Accept / reject logic:

- If $\epsilon \le$ tol: accept $y_{n+1} = y^{(5)}$, append T, X, V, H; set $h \leftarrow h_{\text{new}}$.
- If $\epsilon >$ tol: reject the step, set $h \leftarrow h_{\text{new}}$ and recompute the same step (no time advancement).

Storage (matching Python lists):

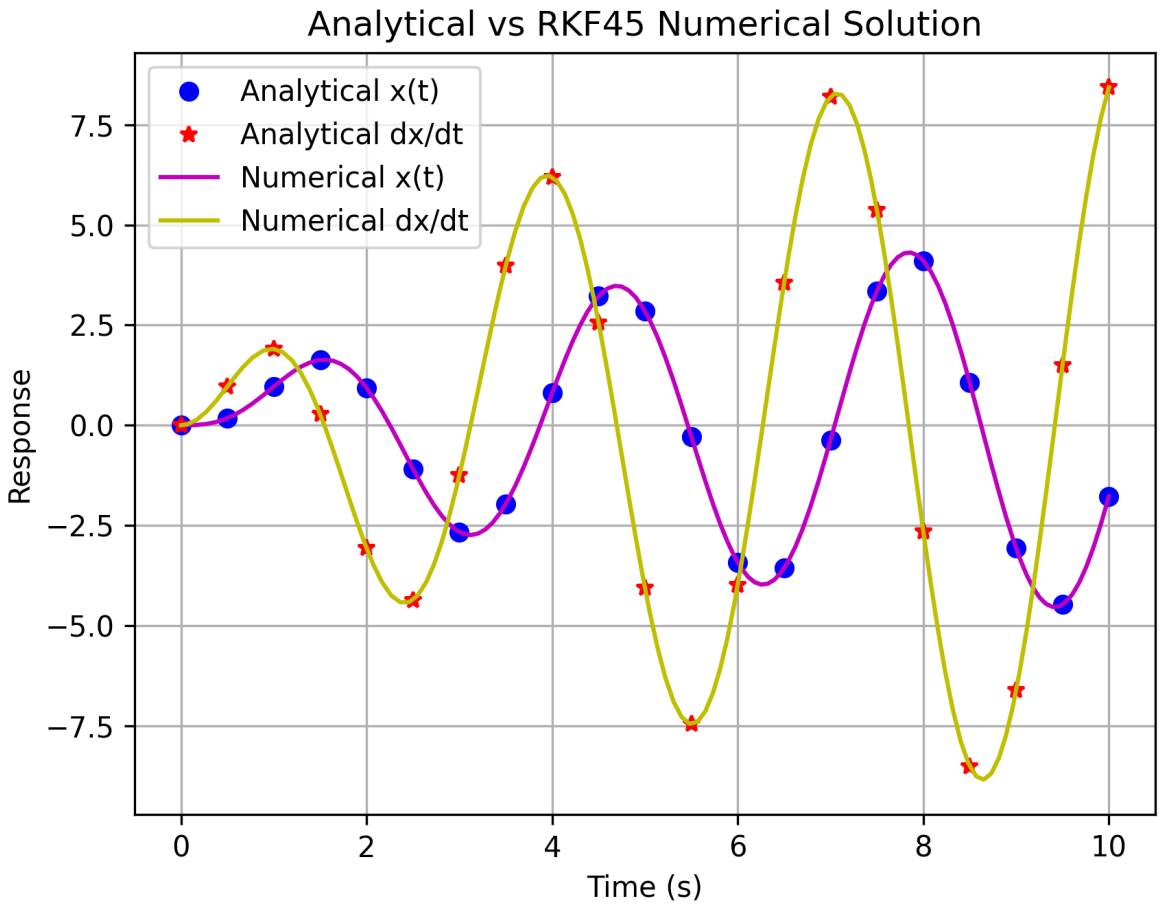
$$T \leftarrow \{\, t_0, t_1, \dots \},\quad X \leftarrow \{\, x(t_i) \,\},\quad V \leftarrow \{\, v(t_i) \,\},\quad H \leftarrow \{\, h_i \,\}.$$

Error Comparison: RK4 vs. RKF45



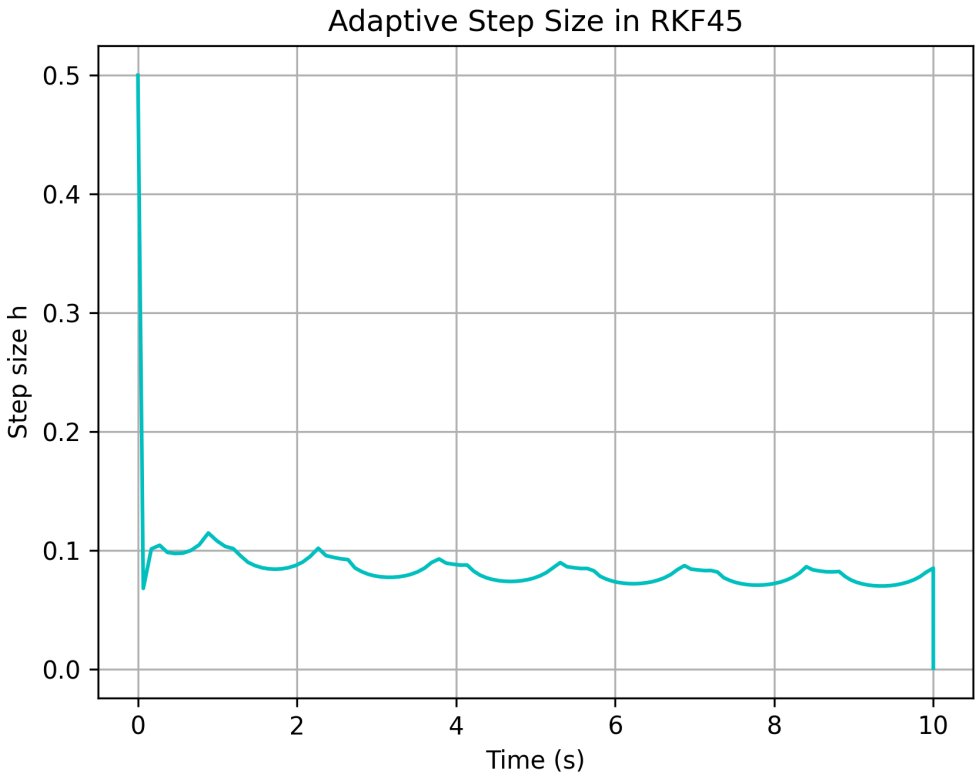
Explanation: RK4's fixed step size accumulates truncation error, causing growing phase drift. RKF45 uses adaptive step control, comparing 4th/5th-order solutions to estimate local error. It dynamically adjusts step size to maintain accuracy within the specified tolerance, preventing long-term error growth and demonstrating superior stability for oscillatory systems like mass-spring-damper simulations.

Analytical vs Numerical Solution



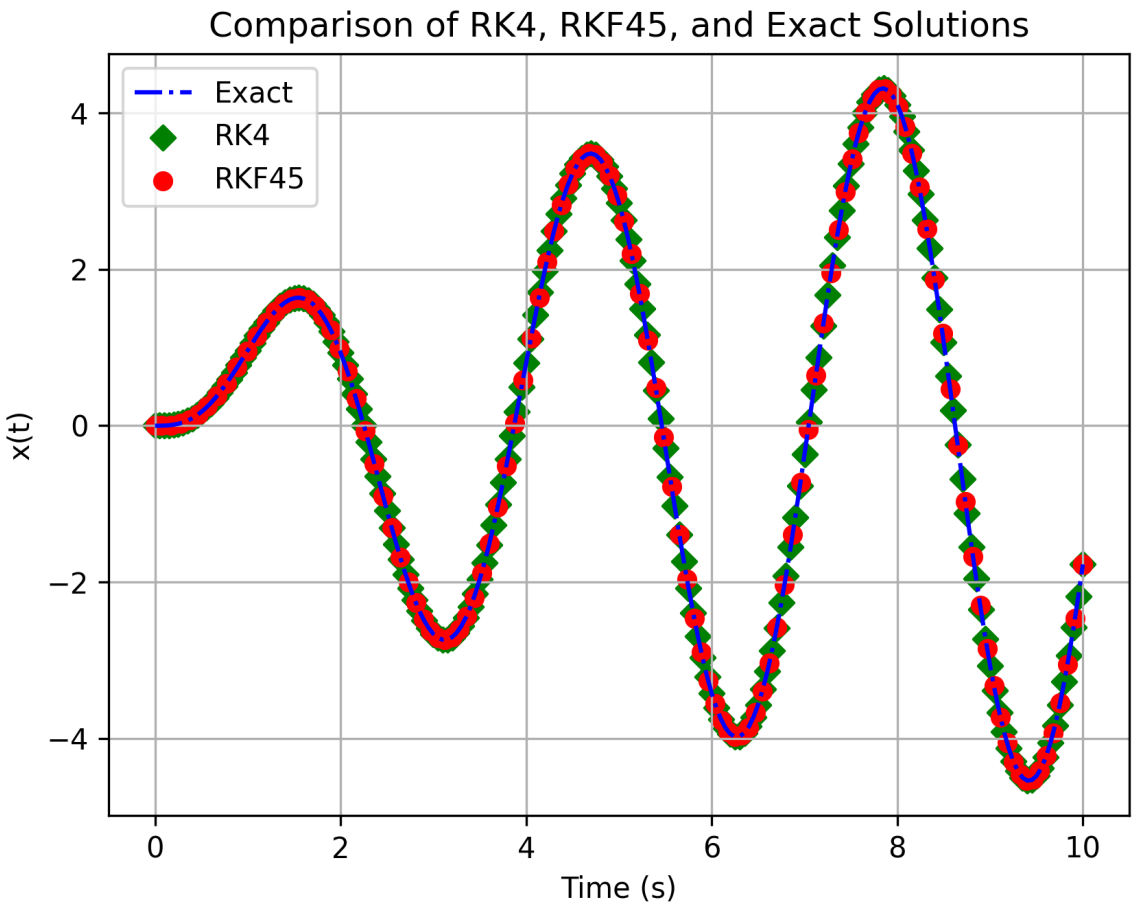
Explanation: We observe two different trajectories because we are plotting both **displacement** $x(t)$ and **velocity** $dx/dt(t)$. The initial transient mismatch occurs as the numerical solution converges. The RKF45 method then rapidly achieves excellent agreement with the analytical results for both state variables in the steady state, validating the solver's accuracy in capturing the full system dynamics.

Adaptive Step Size



Explanation: The step size adapts based on local error estimates. The **sharp peak at t=0** occurs because the initial conditions are simple (zero), allowing a large first step. It then **oscillates** as the solver reduces h to capture rapid system dynamics and increases it in smoother regions. The **final drop at t=10** is a standard correction to hit the end-time exactly without overshooting.

Comparison of All Solutions



Explanation: The point density reveals the solver strategy. **RK4** uses a fixed time step, creating a uniform point distribution. **RKF45** adapts: it places more points in high-curvature regions (peaks/troughs) where the solution changes rapidly, ensuring accuracy. In straight, predictable regions, it uses fewer, larger steps. This intelligent sampling allows RKF45 to closely follow the analytical solution with high efficiency.

Numerical Table (First 7 Samples)

Iter	t	RK4	RKF45	Exact
1	0.00	0.000000	0.000000	0.000000
2	0.05	0.000207	0.000517	0.000207
3	0.10	0.001640	0.000517	0.001639
4	0.15	0.005472	0.007802	0.005472
5	0.20	0.012800	0.007802	0.012800
6	0.25	0.024623	0.031937	0.024622
7	0.30	0.041821	0.031937	0.041821

Observation: The table reveals RKF45's adaptive logic. At t=0.05, RKF45 takes a large step, jumping to a future state (hence the value 0.000517 vs. the exact 0.000207). It then holds this value constant until its next calculation at t=0.15, creating a "stair-step" pattern. This shows the solver prioritizing efficiency in smooth regions. In contrast, RK4, with its small, fixed step size, calculates at every 0.05s interval, closely tracking the exact solution point-by-point in this initial phase. This demonstrates the fundamental trade-off: RK4's immediate local accuracy versus RKF45's efficient, long-term error control.

Performance Comparison

Quantity	RK4	RKF45
Total steps	201	124
Max error	5.3483	0.01125
CPU time (s)	0.0012	0.0012
Step type	Fixed	Adaptive

Observation: RKF45 achieves superior efficiency and accuracy. It uses **38% fewer steps** than RK4 while reducing the maximum error by a factor of **475**. The similar CPU time highlights that adaptive step computation is cost-effective. This demonstrates that intelligent step-size control is far more impactful than simply using more steps, making RKF45 ideal for long-term simulations where error accumulation is critical.

Discussion & Conclusion

- **Adaptive Superiority:** RKF45's dynamic error control mechanism enables it to maintain consistent accuracy throughout the simulation, while RK4's fixed-step approach suffers from accumulating phase errors in oscillatory systems.
- **Efficiency Analysis:** The adaptive strategy proved remarkably efficient - RKF45 achieved superior accuracy with only 124 steps compared to RK4's 201 steps, demonstrating intelligent computational resource allocation.
- **Error Performance:** The dramatic difference in maximum error (5.3483 for RK4 vs. 0.01125 for RKF45) highlights the critical importance of error monitoring and control for long-term simulation stability.
- **Practical Implications:** For engineering applications involving oscillatory systems like mass-spring-damper configurations, adaptive solvers like RKF45 provide the optimal balance between computational efficiency and numerical accuracy, making them preferable for reliable long-term dynamics prediction.

Key Finding: RKF45 reduces maximum error by 475x while using 38% fewer computational steps than classical RK4.

Future Work

- **Nonlinear Dynamics:** Extend analysis to nonlinear springs (Duffing oscillator) and quadratic damping models to study more complex physical behaviors beyond linear system approximations.
- **Resonance Phenomena:** Conduct systematic resonance studies by sweeping forcing frequencies ω near natural frequency ω_n, analyzing amplitude response and phase relationships under varying forcing amplitudes.
- **Advanced Solvers:** Implement higher-order adaptive methods (RKF78, DOPRI8) for increased precision in sensitive applications, and compare their computational cost-accuracy trade-offs against RKF45.
- **Stiff Systems:** Explore strongly damped regimes where explicit methods become unstable, requiring implicit solvers (Backward Euler, Rosenbrock methods) for numerical stability.
- **Multi-body Systems:** Scale the approach to coupled spring-mass systems and multi-degree-of-freedom structures to investigate modal interactions and energy transfer phenomena.

References

N. Ching Kok and S. Mohd Puzi, "Adaptive Runge Kutta of Fehlberg Method on Displacement of Spring due to Opposite Force," *Proceedings of Science and Mathematics*, vol. 10, pp. 200–209, 2022.

https://www.scribd.com/document/838421121/4D-runge-kutta-and-adaptive-step-size

https://files.core.ac.uk/download/pdf/322471429.pdf

Thank You!

Questions are welcome.